

Data Engineering Case Study

Background

Thousands of trades are generated and transmitted to a central store daily. The organization wants to modernize its data infrastructure to support real-time analytics, compliance, and reporting. You are tasked with designing and implementing a robust, scalable ETL pipeline that ingests, processes, validates, and stores trade data using cloud-native tools.

Requirements

Data Ingestion

- Simulate trade data generation (using a script or a mock service).
- Load into Snowflake.

Data Processing & Validation

- Use a Snowflake features data processing engine to consume trades.
- Apply business rules using DBT. Below are independent requirements:
 1. Reject trades with a lower version than existing.
 2. Replace trades with the same version.
 3. Reject trades with a maturity date earlier than today.
 4. Mark trades as expired if the maturity date has passed.
 5. If your familiar with trade processing – you can add your own rules (Optional)
 6. Log rejected trades for audit.

Data Storage

- Store valid trades in a cloud data warehouse - Snowflake.
- Store rejected trades in a separate table for compliance.

Orchestration & Monitoring

- Use any open source tool for orchestration.
 1. Schedule batch jobs (if needed).
 2. Monitor pipeline health and failures.
 3. Implement basic alerting (e.g., email on pipeline failure).
 4. Error handling for DBT.

Optional Enhancements

- Create a dashboard/report to visualize trade status (expired, valid, rejected).
- Document your architecture with diagrams (PlantUML or similar).
- Use Airflow/Composer to orchestrate the ETL workflow.

Deliverables

- Source code/scripts for data generation, ETL pipeline, and orchestration.
- Infrastructure-as-Code (IaC) scripts (Terraform or Deployment Manager) for provisioning resources (optional).
- Documentation:
 1. Architecture diagram.
 2. Step-by-step setup and execution guide.
 3. Description of validation logic and tech stack choices.
- Store all code and documentation in a public GitHub repository.
- Implement CI/CD pipeline to validate and deploy DBT and Terraform code.
- How would you handle potential issues like file arrival delays, data quality problems, or task failures?
- How would you monitor the pipeline's health and performance using Snowflake's administrative views and how would you set up Alerts?
- Scalability: How would your design scale if the number of trades or files increases by 10,000x?

Preferred Tech Stack

- Ingestion: Snowflake Native
- Processing: Snowflake and DBT
- Orchestration: Airflow/Composer/Snowflake Native
- Storage: Snowflake
- Visualization: Looker Studio, Tableau, Stream lit or similar (optional)
- IaC: Terraform / Alternate compatible for Snowflake

Evaluation Criteria

- Correctness and completeness of ETL logic.

- Use of appropriate cloud-native tools and Snowflake feature.
- Code quality, modularity, and documentation.
- Monitoring, error handling, and alerting.
- Clarity of architecture and design decisions.

Good Luck!!!